

tensorflow.keras.preprocessing.image.ImageDataGenerator

- Pre-process (e.g. rescale pixel values) and augment (e.g. rotate) images
- Assign binary labels to training set, using `ImageDataGenerator(...).flow_from_directory`.
- Set up data generators that read pictures in the source folders, convert them to float32 tensors, and feed them (with their labels) to a neural network
- There will be one generator for the training images and one for the validation images
- The ImageDataGenerator class allows to instantiate generators of *augmented* image batches, and their labels via `.flow(data, labels)` or `.flow_from_directory(directory)`
- These generators can then be used with the Keras model methods that accept data generators as inputs: `fit`, `evaluate_generator`, and `predict_generator`

Augmentation with ImageDataGenerator

- `rotation_range` : Int. Degree range for random rotations.
- `width_shift_range` : float: fraction of total width, if < 1, or pixels if >= 1.
- `height_shift_range` : float: fraction of total height, if < 1, or pixels if >= 1.
- `brightness_range` : Tuple or list of two floats. Range for picking a brightness shift value from.
- `shear_range` : Float. Shear Intensity (Shear angle in counter-clockwise direction in degrees)
- `zoom_range` : Float or [lower, upper]. Range for random zoom. If a float, [lower, upper] = [1-zoom_range, 1+zoom_range].
- `channel_shift_range` : Float. Range for random channel shifts.
- `horizontal_flip` : Boolean. Randomly flip inputs horizontally.
- `vertical_flip` : Boolean. Randomly flip inputs vertically.
- `fill_mode` = 'nearest'

Typical workflow

1. Define `ImageDataGenerator` separately for the training and validation sets.
Rescale pixel values from 0-255 to between 0 and 1 using `rescale=1./255` because the pixel range 0-255 is too high for the model to process (given a typical learning rate).
Choose a `validation_split` (e.g. 0.2).
Augment images *in the training set only*.

```
train_datagen = ImageDataGenerator(rescale=1./255,  
    rotation_range=40,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    fill_mode='nearest',  
    validation_split = 0.2)
```

```
valid_datagen = ImageDataGenerator(rescale=1./255,  
    validation_split = 0.2)
```

2. Take the path to a directory and generate batches of augmented data.
Use the *same seed* for both train and validation generators to avoid leakage.

```
train_generator = train_datagen.flow_from_directory(train_dir, batch_size=20,  
class_mode='binary', target_size=(input_width, input_height), subset = 'training', seed = 42)
```

```
valid_generator = valid_datagen.flow_from_directory(train_dir, batch_size=20,  
class_mode='binary', target_size=(input_width, input_height), subset = 'validation', seed =  
42)
```

In each of the resulting generators, the first index labels the batch number (which batch we are in). Each batch is a 2-tuple, where the first object contains the processed images, and the second contains the one-hot encoded labels.

3. To train a model, use

```
history = model.fit(train_generator, validation_data = valid_generator, epochs = 40)
```

4. Model predictions:

```
model.evaluate(valid_generator) , model.predict(test_preprocessed) > 0.5).astype(int) , etc.
```